Object Oriented Programming (CSEG1044)
Unit 01 – Lecture 02 Notes
Java Overview + Program Structure + Features

Tofik Ali

February 26, 2026

Contents

1 Lecture Overview **1**

2 Syllabus Alignment (Unit 01) **2**

3 Learning Outcomes **2**

4 Key Terms (Quick) **2**

5 Detailed Notes **2**
5.1 Java ecosystem: JDK vs JRE vs JVM . . . 2
5.2 Bytecode and the compile/run flow . . . 3
5.3 Basic program structure: class + `main` . . . 3
5.4 Compile-time vs runtime errors . . . 3
5.5 Features of Java (what they mean in practice) . . . 3

6 Worked Example: Compile & Run + Observe Errors **4**

7 In-Class Exercises (with brief solutions) **4**

8 Self-Study Practice (no solutions) **5**

9 Checkpoint Questions (with sample answers) **5**

1 Lecture Overview

This lecture makes you comfortable with the Java "toolchain" and the minimum you need to compile and run Java programs. By the end, you should clearly understand:

- what the **JDK**, **JRE**, and **JVM** mean,
- how **bytecode** enables Java to run on different platforms,
- the difference between **compile-time errors** and **runtime errors**.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed February 26, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed February 26, 2026).

2 Syllabus Alignment (Unit 01)

Unit 01: Introduction to OOPs (Java overview + features)

3 Learning Outcomes

After this lecture, you should be able to:

- Explain the Java ecosystem: JDK vs JRE vs JVM, and what bytecode is.
- Compile and run a Java program using `javac` and `java`.
- Identify whether an error is compile-time (during `javac`) or runtime (during `java`).
- List core features of Java and what they imply in practice.

4 Key Terms (Quick)

- **JDK (Java Development Kit)**: tools for development (compiler `javac`, docs, etc.).
- **JRE (Java Runtime Environment)**: runtime needed to run Java apps (JVM + libraries).
- **JVM (Java Virtual Machine)**: executes Java bytecode (`.class`).
- **Bytecode**: platform-neutral instructions produced by `javac`.
- **Classpath**: where the JVM looks for compiled classes and libraries.
- **Compile-time error**: detected by the compiler before running.
- **Runtime error/exception**: occurs while the program is executing.

5 Detailed Notes

5.1 Java ecosystem: JDK vs JRE vs JVM

Think of Java in layers:

- **JDK**: used by developers to *create* programs. Includes `javac` (compiler) and other tools.
- **JRE**: used to *run* programs. Includes the JVM and standard libraries.
- **JVM**: the engine that loads classes, verifies bytecode, and executes it.

Rule of thumb. If you want to *write/compile* Java, install the **JDK**. If you only want to *run* Java (rare for students), the **JRE** is enough.

5.2 Bytecode and the compile/run flow

Java is often described as "Write Once, Run Anywhere" because compilation produces **bytecode**, not machine code. The same `.class` file can run on Windows/Linux/macOS *as long as there is a JVM for that OS*.

Flow (mental model):

1. Write: `Hello.java`
2. Compile: `javac Hello.java` → `Hello.class`
3. Run: `java Hello` (JVM loads `Hello.class` and starts `main`)

5.3 Basic program structure: class + `main`

A minimal program has:

- a **class** (container for code),
- the **entry point** method: `public static void main(String[] args)`.

Important rule: if you declare a `public` class, the file name must match it exactly. Example: `public class Hello` must be in `Hello.java`.

5.4 Compile-time vs runtime errors

Compile-time error examples:

- syntax errors (missing semicolon, wrong braces),
- type errors (assigning `String` to `int` without conversion),
- missing symbols (calling a method that doesn't exist).

Runtime error/exception examples:

- `ArithmeticException` (divide by zero),
- `NullPointerException` (calling method on `null`),
- `ArrayIndexOutOfBoundsException` (invalid index).

5.5 Features of Java (what they mean in practice)

- **Platform independent**: bytecode runs on any platform with a JVM.
- **Object-oriented (mostly)**: you model problems using classes/objects (but primitives exist for efficiency).
- **Automatic memory management**: garbage collector reclaims unused objects (no manual `free`).
- **Strong typing**: many errors are caught at compile time (good for large codebases).

- **Rich standard library**: collections, I/O, networking, GUI, database connectivity, etc.
- **Security model**: class loading + bytecode verification help reduce unsafe behavior.

Sources: Schildt, *Java: A Beginner's Guide* (10e), McGraw-Hill (Java fundamentals + OOP basics).; Oracle Java Tutorials: Getting Started + Language Basics + Classes and Objects (accessed February 26, 2026).; Java SE API docs (e.g., `String`, `Scanner`) (accessed February 26, 2026).

6 Worked Example: Compile & Run + Observe Errors

A minimal program

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

How to run (terminal)

- Compile: `javac Hello.java`
- Run: `java Hello`

Runtime exception example

```
public class DivideDemo {
    public static void main(String[] args) {
        int a = 10;
        int b = 0;
        System.out.println(a / b); // ArithmeticException at runtime
    }
}
```

Compile-time error example (explain in class)

If you write `System.out.println("Hi")` without a semicolon, `javac` will stop and show an error. This is a compile-time error: the program never runs.

7 In-Class Exercises (with brief solutions)

1. **JDK/JRE/JVM:** Which one contains `javac`? **Answer:** JDK.
2. **File naming:** What should the file name be if you write `public class Student`? **Answer:** `Student.java`.
3. **Error type:** Is "cannot find symbol" a compile-time or runtime error? **Answer:** compile-time.

8 Self-Study Practice (no solutions)

1. Write `ArgsDemo` that prints all command line arguments with index numbers.
2. Create 2 small programs: one that fails at compile time, one that fails at runtime. Write a 2-line explanation for each.
3. Install the JDK and run `java -version` and `javac -version`. Note the output.

9 Checkpoint Questions (with sample answers)

Q1. Why does Java use bytecode instead of compiling directly to machine code?

Sample answer: Bytecode makes the compiled program portable. The same `.class` can run on different operating systems because the JVM translates/executes bytecode for the current platform.

Q2. What is the difference between a compile-time error and a runtime exception?

Sample answer: Compile-time errors are detected by `javac` before execution; runtime exceptions occur while the program is running after compilation succeeds.